

Logistics

Data science: Interdisciplinary field about processes and systems to extract knowledge or insights from data in various forms.

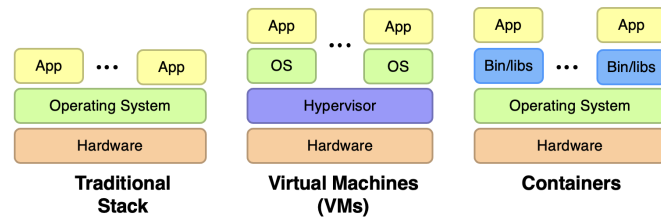
- **Processing** diff. types of data (large data volume, graph data, stream data).
- **Models of Computation:** MapReduce/Spark, Large graph processing engines, Streams and online algorithms.

Cloud Computing Infrastructure

Cloud computing: scalable, elastic infrastructure for big data systems.

Utility Computing:

- Computing resources as a metered service ("pay as you go").
- Ability to dynamically provision virtual machines.
- **Scalable:** ("Infinite" capacity).
- **Elastic:** (Scale up or down on demand).



Virtual Machines: Enable sharing of hardware resources, run each app in isolated virtual machine.

- High overhead as each VM has its own OS.

Containers: Software package enable lightweight sharing of resources.

- Apps run in isolation but share same OS.
- Encapsulates an app and environment.

Everything as a Service

- **Infrastructure (IaaS):** Utility Computing. User rents a virtual machine decide what to run on it. (e.g. Amazon EC2).
- **Platform (PaaS):** Provide hosting web apps. Cover hardware maintenance, upgrades. (e.g. Google App Engine. User provides their web app. and sys. takes care of all details for hosting it.)
- **Software (SaaS):** User typically doesn't write code, and is just using an existing app. (e.g. Gmail, Dropbox, Zoom)

Principles of Big Data Systems

Data Center Infrastructure

- **Single Server:** A commodity hardware unit.
- **Rack:** Dozens of servers, connect to rack switch (top-of-rack switch).
- **Data Center:** Hundreds of racks, connect to data center switch (core switch).

Key Idea: The datacenter is the computer. (The computers in a data center as one "big computer" being used to perform a task, rather than just as many independent computers)

Bandwidth: Max amt of data transmitted per unit time (e.g. GB/s).

- Approximately the **minimum** bandwidth along the path.
- Measurement of transmitting time for **large** amount of data
- Bandwidth and Throughput affect file transfer speed (esp. for large files)

Throughput: Rate data is **actually transmitted** in network in given time period.

- Similar to bandwidth, but refer to **capacity**

Latency: Time taken one packet travel src to dest (one-way) / round-trip (in ms).

- Latency combines approximately additively. $L_{\text{total}} = \sum L_i$
- Measurement of transmitting time for **small** amount of data
- Affects delay when communicating,

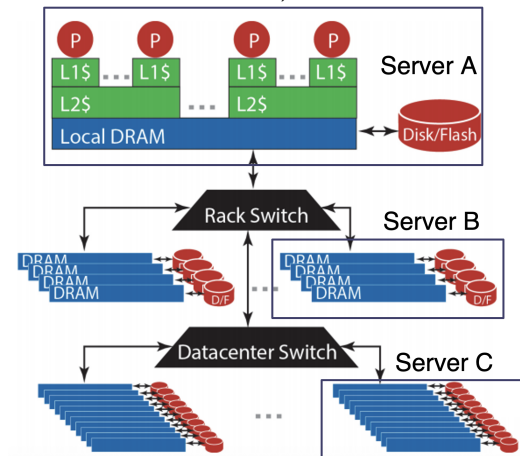
Storage Hierarchy: based on capacity, latency, bandwidth.

- **DRAM:** Fast but limited capacity.
- **Flash:** Intermediate between DRAM and Sisk.
- **Disk:** Slow but large capacity. Reads more expensive than DRAM (higher latency, lower bandwidth).

Data Flow Paths

Data flow paths depend on the location of the source and destination. Data access may cross several components, e.g. hard disk, DRAM, rack switch, cluster switch.

1. Local DRAM → Local Disk / Local DRAM in same server.
2. Server A DRAM to server B DRAM in same rack: Local DRAM → Rack Switch → Server B's DRAM
3. Transfer across **different** rack, **same** cluster: Local DRAM → Rack Switch → Datacenter Switch → Server C's Rack Switch → Server C's DRAM.
4. Load from disk, usually need to load into DRAM first. (Local Disk → Local DRAM → Rack Switch)



Properties and Trends

- Disk has much higher capacity than DRAM
- **Capacity** ↑, **Costs** (Latency ↑, bandwidth ↓) ↑ from local to rack to data center, along storage hierarchy.
- Bandwidth tends to be **bottlenecked** by networking switches: ↓ when we need to pass through these switches (while transferring to disk has less effect in these cases)
- Latency is dominated by disk speed, any transfers involving disk take about the same amount of time

Building Efficient Big Data Systems

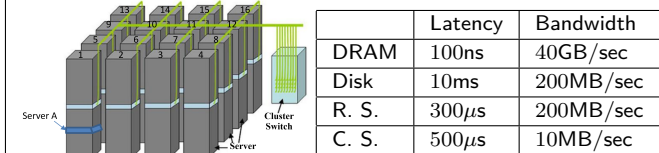
1. **Scale "Out", Not "Up":** Combine many **cheaper** machines (out, horizontal Scaling) instead of increase power of individual machine (up, vertical scaling).

2. **Seamless Scalability:** Ideal (linear) scalability, performance ↑ proportionally to added resources. (without being disruptive)
3. **Move Processing to the Data:** As clusters limited bandwidth; move tasks to machine where data is stored.
4. **Process Sequentially Not Random:** Disk seeks expensive (high latency); sequential access data provides better throughput.

Challenges in Big Data Systems

1. **Machine Failures:** With enough scale, expect daily machine failure.
2. **Synchronization:** Difficult due to unknown order of worker execution, interruptions, and shared data access. (Use semaphores, conditional variables, and barriers to address)
3. **Programming Difficulty:** Concurrency hard to reason about, especially at scale. Debugging and managing failures add complexity.

Example Big Data System



Suppose a program P is running on Server A in Rack 1.

1. **Throughput Analysis:** Define

- T_1 : Time of P reading a chunk of 128MB from a file in the hard disk of Server A

T_1 is bounded by 200MB/sec,

$$T_1 \approx \frac{128\text{MB}}{200\text{MB/s}} = 0.64\text{s}$$

- T_2 : Time of P reading a chunk of 128MB from a file in the hard disk of the other server in Rack 1

T_2 is bounded by 200MB/sec,

$$T_2 \approx \frac{128\text{MB}}{200\text{MB/s}} + 10\text{ms} + 2 \times 300\mu\text{s} \approx 0.65\text{s}$$

- T_3 : Time of P reading a chunk of 128MB from a file in the hard disk of the other server in Rack 16

T_3 is bounded by 10MB/sec,

$$T_3 \approx \frac{128\text{MB}}{10\text{MB/s}} = 12.8\text{s}$$

Therefore $T_1 \approx T_2 \ll T_3$

2. **Latency Analysis:** Define the latency of P accessing

- L_1 : a byte in the hard disk of Server A
 $L_1 \approx 10\text{ms}$
- L_2 : a byte in the hard disk of other servers in Rack 1
 $L_2 \approx 10\text{ms} + 2 \times 0.3\text{ms} = 10.6\text{ms}$
- L_3 : a byte in the hard disk of other server in Rack 16
 $L_3 \approx 10\text{ms} + 2 \times 0.3\text{ms} + 2 \times 0.5\text{ms} = 11.6\text{ms}$

Since hard disk is too slow, $L_1 \lesssim L_2 \lesssim L_3$

3. When should we use throughput and when latency?

Answer: We should use throughput when the data is big (i.e. 128MB), and focus on latency when the data is small (i.e. 40KB)

MapReduce

- prog. model & associated runtime system to process large datasets in parallel across distributed cluster. (Parallel + Divide & Conquer Paradigm).
- Provides functional abstractions for two operations.
- Optionally specify **partition**, **combine** functions, optimize, reduce network traffic.

Map: $(k1, v1) \rightarrow \text{List}(k2, v2)$.

- Process each input K-V pair and emits intermediate K-V pairs.

Reduce: $\text{reduce}(k2, \text{List}(v2)) \rightarrow \text{List}(k3, v3)$.

- Aggregate intermediate results to generate the final output.
- All values with same key sent to same reducer.

Partition: runs in local write phase (which keys to which reducer).

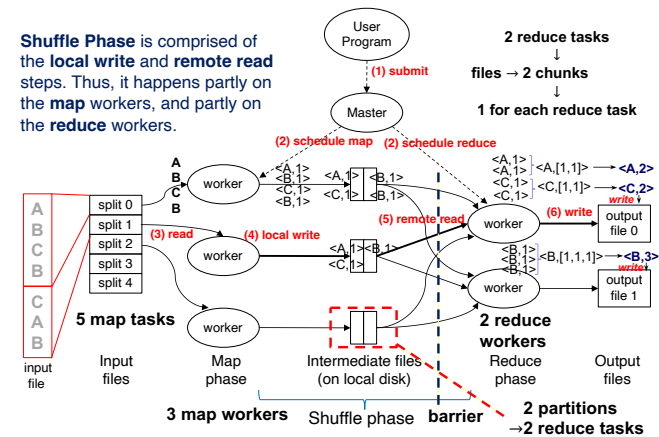
- By default, keys assigned to reducers using hash function.
- Implement custom partitioner to balance load among reducers. (E.g. if some keys much more values than others).

Combiner: Locally **aggregate** map outputs to ↓ disk and network I/O.

- Runs after map phase, done during local write phase, **before** actual disk write.
- Reduce # of tuples emitted $\Rightarrow$ save disk and memory I/O
- The combiner function must produce the same type of intermediate output as the mapper output
- Correctness:** Must not affect correctness of final output, regardless of how many times it runs (include 0 times)
 - sum, max holds correctness
 - mean, minus **do not** holds correctness

Hadoop Implementation

Shuffle Phase is comprised of the **local write** and **remote read** steps. Thus, it happens partly on the **map** workers, and partly on the **reduce** workers.



- Submit:** user submits MapReduce program (including code for the map and reduce functions), configuration (e.g., no. of workers, no. of reduce tasks) to Master node
- Schedule:** Master schedules resources for map and reduce tasks (Note: Master **does not** handle any actual data)
- Read:** Input files are separated into “splits” of around 128MB.
 - Each split corresponds to one “map task”, which is the work that needs to be done to process this split.
 - Workers execute map tasks 1 at a time.
- Map phase:** Each worker iterates over each $\langle \text{key}, \text{value} \rangle$ tuple in its input split, and computes the map function on each tuple

Local write:

- Each worker partitions the map outputs using the partitioner
 - Sorts each partition by key
 - Writes the results to intermediate files on its local disk.
5. **Remote read:** Each reduce worker is responsible for ≥ 1 keys.
- For each such key, it reads the data it needs from the corresponding partition of each mapper's local disk
 - Merge-sorts them (an efficient way of merging multiple sorted lists into a single sorted list)
 - Groups tuples with the same key to form lists.
6. **Reduce phase:** The reducer receives all its needed key value pairs, where the keys arrive in **sorted** order. It computes the reduce function on the values of each key.
7. **Write:** The output of the reduce function is written (usually to HDFS, a distributed file system)

Notes and Other Optimizations

- Execution Time** is determined by
 - system's ability to read from disk
 - transfer data over the network
 - efficiency of distributing the workload across the cluster
- Size of split (or chunk)**
 - Too big $\rightarrow$ limited parallelism
 - Too small $\rightarrow$ high overhead (master node may be overwhelmed by scheduling work) $\Rightarrow$ may outweigh benefits of parallelism
- Worker:** single component of cluster, performs storage, processing tasks. Can handle multiple map tasks (re-assign after completing)
- Map Task:** A basic unit of work, typically 128MB.
 - Map() $\rightarrow$ Partition $\rightarrow$ Sort $\rightarrow$ Combine $\rightarrow$ Write to local disk
 - a single map task can involve many calls to such a map function
 - a **job** requiring to process one split, **not a worker**.
 - #map fn calls = # input key-value pairs. # map tasks = # input splits.
- Reduce Task:** Process a subset of intermediate keys.
 - Remote read $\rightarrow$ Merge-sort $\rightarrow$ Reduce()
 - a single reduce task can involve many calls to reduce function
 - if handles multiple keys, it will process these keys in sorted order
 - Reduce function calls = no. of distinct intermediate keys.

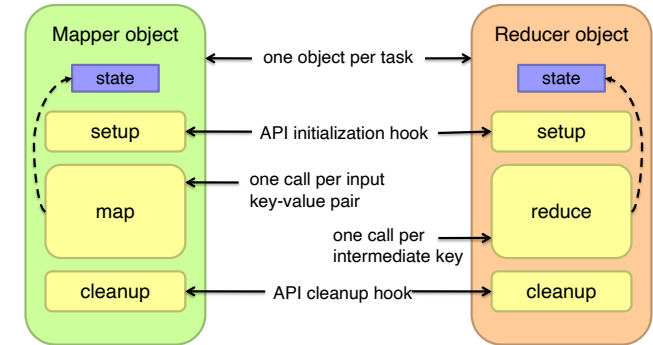
Summary Number of Tasks and Worker Analysis

Suppose program is running on a cluster of 10 machines (each with a single CPU core)

- Reducer:** 10^5 Map tasks and Y Reduce tasks
 - 1~10: better performance due to parallelism.
 - 10~X0: better perf. due to multi-tasking on same machine.
 - X0~ 10^5 : perf. become stable or even ↓ due to saturation.
 - Mapper:** Similarly, from 1 to 10^5 throughput: Bad $\rightarrow$ Good $\rightarrow$ Stable $\rightarrow$ Degrade (Too much shuffle)
 - Input:** Fixed Mapper/Reducer #, input size↑
 - Small input to a certain amount: throughput ↑ since more machines are used and parallelism improves
 - After that, throughput becomes rather stable. The disk and network bandwidths of all machines are already fully utilized
- Barrier:** between map and reduce phases.
 - Otherwise, reduce phase may compute wrong answer.
 - Shuffle phase can begin copying intermediate data earlier, but only from mappers which have completed their task

(due to the sort and merge-sort steps in the shuffle phase)

- Preserving State in Map / Reduce Tasks:** In MapReduce, state variables can be used to store information that persists across **multiple** calls to map() or reduce() functions within same task. Sharing data or resources between function calls.



General Performance Guidelines (Basic Algo. Design)

- Linear Scalability:** More nodes should enable more work done in same time.
- Minimize Disk I/O:** Sequential access over random access.
- Minimize Network I/O:** Send data in bulk over small chunks.
- Reduce Memory Working Set:** Avoid large in-memory data structures to prevent out-of-memory errors.

Secondary Sort: Use this property to sort values associated with key. (reduce work done in reducer by using inbuilt distributed sorting process of Hadoop).

- Combine key and value into composite key e.g. (UserID $\rightarrow$ [UserID-Timestamp])
- Define **custom** comparator (sort by ID then timestamp) & **partitioner** (group by ID only) $\Rightarrow$ ensure that tuples with the same value of natural key partitioned to same reduce task $\Rightarrow$ we can still complete tasks which require accessing all tuples for a given key.

Word Count Algorithm

Version 1: Emit for each line. $I/O \propto \max\{\# \text{ of distinct words / line} \}$

```
1 class Mapper {
2   def map(key: Long, value: Text) = {
3     val counts = new Map()
4     for (word <- tokenize(value)) {
5       counts(word) += 1
6     }
7     for ((k, v) <- counts) {
8       emit(k, v)
9     }
10  }
11 }
```

Version 2: Emit for entire file. $I/O \propto \# \text{ of distinct words}$

```
1 class Mapper {
2   val counts = new Map()
3   def map(key: Long, value: Text) = {
4     for (word <- tokenize(value)) {
5       counts(word) += 1
6     }
7   }
8   def cleanup() = {
9     for ((k, v) <- counts) {
10      emit(k, v)
11    }
12  }
```

```

11     }
12 }
13 }

```

- Use a hash table to maintain the words and **counts across all lines** in a single split $\Rightarrow$ $\downarrow$ disk and memory I/O more than version 1
- Preserves state in the Mapper object
- A drawback is increasing the memory working set (proportional to the number of distinct words in a map task)

Regular Combiners vs In-Memory Combiners

- In some cases, using in-mapper combiners could lead to out of memory errors, so in that case regular combiners should be used.
- Using combiners may be clearer from a code perspective (it communicates the user intent more easily)
- Otherwise, either can be used, and the speedup should be similar, and for the purpose of this class the difference in speed between them can be ignored.
- In practice, there may be slight improved efficiency by using in-mapper combiners since the pairs are not even emitted to the Hadoop framework, but it is not likely to be that significant

Hadoop Distributed File System (HDFS)

Traditional Drawback:

- Moving data is costly in this environment $\Rightarrow$ bottleneck
- Data processing in map/reduce is not very heavy.

Design and Structure

- Files stored as **chunks** (default 128MB) over a set of data nodes
- Reason:** Disk throughput is reasonable, although random access latency is bad.
- Optimize for **high throughput** rather than low latency access $\Rightarrow$ suited for app. with large datasets requiring **batch processing**
- Single name node coordinates access and metadata
- Data locality:** try to read / write data locally
- Write-once, Read-many: $\times$ support modifying files, other than appending

Data Replication

- Each chunk is stored as **multiple replications** (by default 3) to improve fault tolerance.
- Two** replicas in the **same rack**, and **one** in the **other rack**

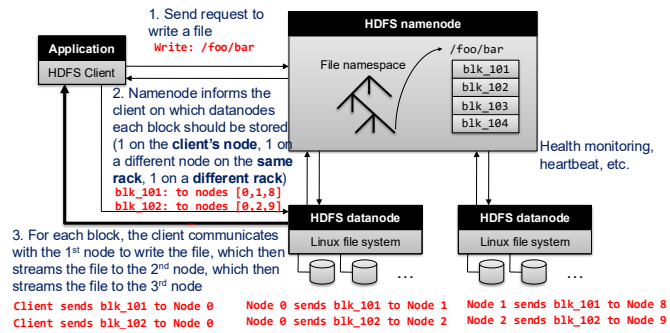
Summary Increase Replication - Pros and Cons

- Pros:** system will be more **robust**. Scheduling is more **flexible**. MapReduce Performance could be better.
- Cons:** Storage overhead and slow write performance.

Assumptions:

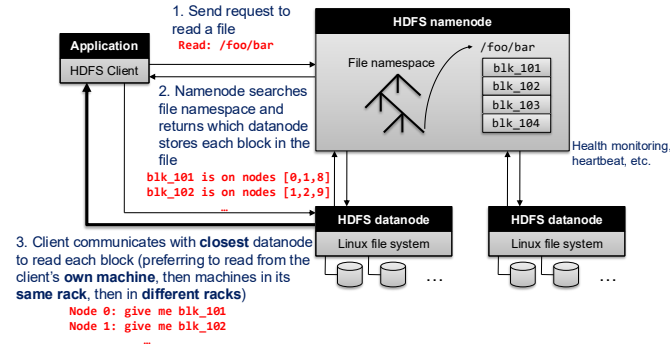
- Commodity hardware instead of exotic hardware (Scale "out") $\Rightarrow$ Data Center Environment $\Rightarrow$ addresses both scale-out (aggregate disk bandwidth to read the data in high speed) and failure.
- High component failure rates: should be resilient
- Modest number of huge files: Large **sequential** reads

Writing Data



- Balances between locality (storing onto the client's local node) and fault tolerance (by replicating on a different node / rack)
- No data passes through namenode
- Replication:** when client writes data to a datanode, the namenode also decides which datanodes should be used as replicas, and same packets are **forwarded** to the 2nd and 3rd nodes in the pipeline

Reading Data



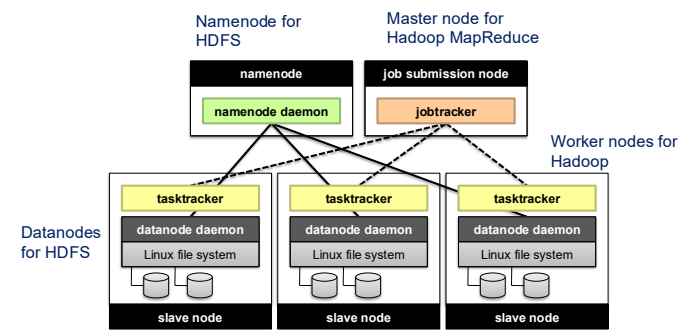
- Each block of file read separately and they can be read in parallel
- Namenode Responsibilities**
 - Managing the file system namespace
 - Holds file/directory structure, metadata, file-to-block mapping, access permissions, etc.
 - Directs clients to datanodes for reads and writes
 - No data is moved through the namenode
 - Maintaining overall health
 - Periodic communication with the datanodes
 - Block rebalancing: ensure data is distributed close to evenly
- If the namenode's data is lost $\Rightarrow$ All files on the filesystem cannot be retrieved since there is no way to reconstruct them from the raw block data. (Fortunately, Hadoop improve resilience through backups and checkpointing.)

Summary Disk I/O Analysis in HDFS

- Job *: reads from HDFS (XGB), and writes the output to HDFS (output Y GB, needs 3 copies, 3Y GB)

Hadoop and Map-Reduce: Both use the same set of nodes.

MapReduce tries to schedule map tasks to run on the machines that already contain the needed data (for data locality) or moving the task to the data



MapReduce and Relational Database

Relational Database

- A relational database is comprised of tables.
- Each table represents a relation = collection of tuples (rows).
- Each tuple consists of multiple fields.

Projection: No reducer needed; No shuffle stage; More Efficient

```

1 def map(key, tuple):
2     remove fields of tuple not in projection set
3     emit tuple

```

Selection: No reducer needed

```

1 def map(key, tuple):
2     if tuple satisfies predicate:
3         emit tuple

```

Group by: Aggregation

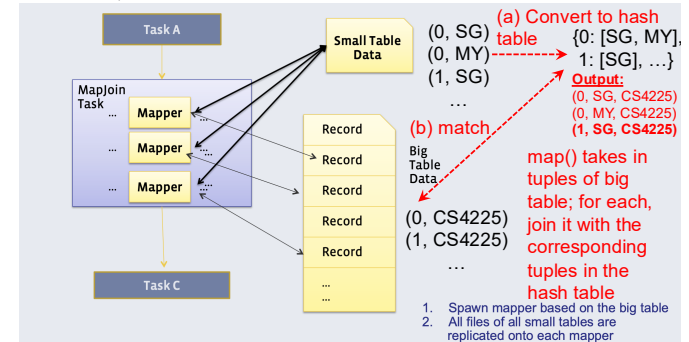
```

1 # SELECT product_id, AVG(price) FROM sales GROUP BY product_id
2 def map(key, tuple):
3     emit <tuple.product_id, tuple.price>
4 def reduce(product_id, prices):
5     emit <product_id, average(prices)>

```

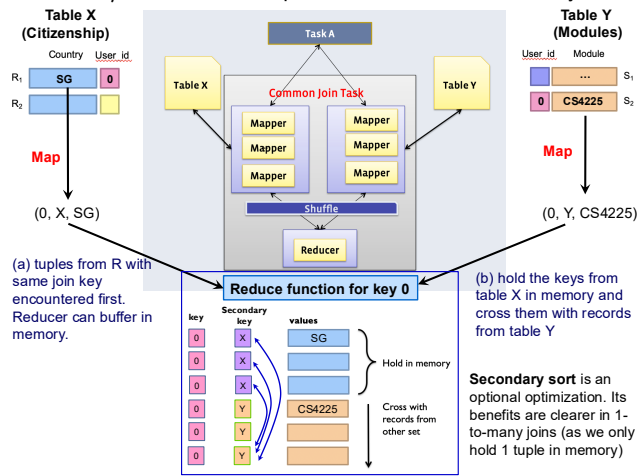
- Map over tuples, emit <product_id, price>
- Framework automatically groups these tuples by key
- Compute average in reducer
- Optional: optimize with combiners

Broadcast/Map Join: Requires one of the tables to fit in memory



- All mappers store a copy of the small table. For efficiency, we convert it to a **hash table**, with keys we want to join by)
- Iterate over big table, and join records with small table
- Not** require a shuffle stage $\Rightarrow$ more efficient than reduce-side join

Reduce-side/Join: Doesn't require a dataset to fit in memory



- Slower than broadcast join
- Different mappers operate on each table, and emit records, with key as the variable to join by
- Reducer:** secondary sort to ensure all keys from table X arrive before table Y $\Rightarrow$ hold the keys from table X in memory and cross them with records from table Y
- Repartition the two datasets by the join key. No particularly efficient since it requires shuffling both datasets across the network.

Similarity Search

Similarity Measures:

- Euclidean distance: $d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{i=1}^D (a_i - b_i)^2}$
- Manhattan distance: $d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_1 = \sum_{i=1}^D |a_i - b_i|$
- Cosine similarity: $s(\mathbf{a}, \mathbf{b}) = \cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$
- Jaccard Similarity: $s_{\text{Jaccard}}(A, B) = \frac{|A \cap B|}{|A \cup B|}$
- Jaccard Distance: $d_{\text{Jaccard}}(A, B) = 1 - s_{\text{Jaccard}}(A, B)$

Finding Similar Documents:

- Shingling: Convert documents to sets of short phrases ("shingles")
 - k-shingle/k-gram:** sequence of k tokens that appears in the doc
Example: $k = 2$; $D = \text{"the cat is glad"} \Rightarrow S(D) = \{\text{"the cat"}, \text{"cat is"}, \text{"is glad"}\}$
 - Each document D_i can be thought of as a set of its k-shingles
 $C_i \Rightarrow \text{sim}(D_1, D_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$
- Min-Hashing: Convert these sets to short signatures of each document, while **preserving similarity**
 - Documents with same signature are **candidate pairs** for finding near-duplicates. Either directly use them as final output, or compare one by one to check if they are actually similar pairs.
 - Fast approximation to the result of using Jaccard similarities to compare all pairs of documents
 - Formally $P[h(C_1) = h(C_2)] = \text{Jaccard-Sim}(C_1, C_2)$
 $\rightarrow \uparrow \text{sim}(C_1, C_2) \Leftrightarrow P[h(C_1) = h(C_2)] \uparrow$
- Extension to multiple hashes:** In practice, use multiple hash functions generate N signatures for each doc. Candidate pairs are those matching a sufficient number.

MapReduce Implementation

Map

- Read over the document and extract its shingles
- Hash each shingle and take the min of them to get the MinHash signature
- Emit $\langle \text{signature}, \text{document_id} \rangle$

Reduce

- Receive all documents with a given MinHash signature
- Generate all candidate pairs from these documents
- (Optional): compare each such pair to check similarity

Clustering

K-Means Algorithm

- Initialization: Pick K random points as centers
- Repeat:
 - Assignment:** assign each point to nearest cluster
 - Update:** move each cluster center to average of its assigned points

Stop if no assignments change

- Always converges
- Final output can depend on the initialization

Reason Different initial cluster centers can lead to different assignments of points in the early iterations $\Rightarrow$ affect subsequent position of new cluster centers and the overall convergence path of the algorithm.

MapReduce K-Means

- $n = \#$ points
- $d = \text{dimensionality}$
- $m = \#$ iterations
- $k = \#$ centers

Version 1: I/O: $O(nmd)$

```

1 class Mapper:
2     def Config():
3         # Reads in current positions of cluster centers
4         cluster = load_clusters()
5     def map(id, point):
6         # Assign each point to closest cluster
7         cluster_id = nearest_cluster_id(cluster, point)
8         p = extend_point(cluster_id, point)
9         emit(cluster_id, point)
10 class Reducer:
11     def reduce(cluster_id, points: []):
12         # Update: computes avg of points in each cluster
13         s = init_point_sum()
14         for point in points:
15             s = s + p
16         centro_id = get_centro_id(s)
17         emit(cluster_id, centro_id)

```

Version 2: I/O: $O(kmd)$

```

1 class Mapper:
2     def config():
3         clusters = load_clusters()
4         H = init_associative_array()
5         return clusters, H
6     def map(id, point, clusters, H):
7         cluster_id = nearest_cluster_id(clusters, point)
8         p_ext = extend_point(point)
9         H[cluster_id] = H[cluster_id] + p_ext
10    def close(H):
11        for cluster_id in H:
12            emit(cluster_id, H[cluster_id])
13 class Reducer:
14    def reduce(cluster_id, points: []):

```

```

15    s = init_point_sum()
16    for p in points:
17        s = s + p
18    centroid = compute_centroid(s)
19    emit(cluster_id, centroid)

```

- V2 Uses an in-mapper combiner to combine points in the same map task using a data structure H.
- Each `emit()` call transmits $O(d)$ bytes of data across the network, since a point p has d dimensions.

Non-Relational Database (NoSQL)

Pros

- + Flexible / dynamic schema:** suitable for less well-structured data
- + Horizontal scalability:** (due to sacrificing strong consistency)
- + High performance and availability:** due to their relaxed consistency model and fast reads / writes (**low latency**)

Cons

- No declarative query language:** query logic (e.g. joins) may have to be handled on the application side, which can add additional programming
- Weaker consistency guarantees:** application may receive stale data that needs to be handled by the application

Flexible Schema: database items need not follow a precise schema

- Traditional RDBMS have **schema-on-write**: data written to the database must follow a schema
- NoSQL is **schema-on-read**: they instead apply the structure of the data when reading or querying

Key-Value Stores (Redis)

- Stores associations between keys and values
- Key:** usually primitives and can be queried
- Value:** primitive or complex, usually cannot be queried
- Simple API:** GET or PUT value associated with key (also have multi-get, multi-put, range queries)
- Suitable for
 - Small continuous read and writes
 - Storing 'basic' information (e.g. raw chunks of bytes), or no clear schema
 - When complex queries are not required / rarely required
- e.g. User sessions, Caching, user data often processed individually

Implementation

- Non-persistent:** Just a big in-memory hash table (Memcached, Redis). But these can also back up the data to disk periodically
- Persistent:** Data stored persistently to disk (RocksDB, Dynamo)

K-V Store

- Improves **scalability** and **efficiency** – writing or reading user pages is **faster**.
- No need for complex queries based on the content of user pages – **just read and write**.
- May be **acceptable** for user pages to be **slightly stale** – then eventual consistency is acceptable

Document Store

- Flexible schema** may be beneficial (e.g. special types of vehicles may require different sets of fields)
- Unlike key-value stores, document stores are more **suitable for queries based on fields of a document**

Document Stores (MongoDB)

- Multiple collections (Collections have multiple documents)
- Document: JSON-like object with fields and values
 - Can be nested (JSON objects as values)
- Flexible** schema to accommodate different doc. characteristics
- Allow some querying based on the content of a document
 - Create

```
1 db.users.insert ({
2   name: "sue", age: 26, status: "A"
3 })
```

– Read

```
1 db.users.find (
2   { age: { $gt: 18 } },
3   { name: 1, address: 1 } // 1 = include
4 ).limit(5)
```

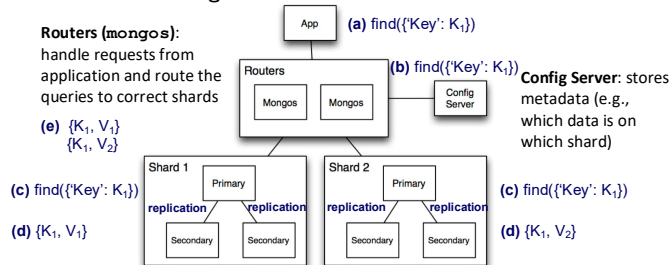
– Update

```
1 db.users.update (
2   { age: { $gt: 18 } },
3   { $set: { status: "A" } }, // UPDATE action
4   { multi: true }
5 )
```

– Delete

```
1 db.users.remove ({ status: "A" })
```

Architecture of MongoDB



Replica sets: multiple processes which maintain the same data, for redundancy

Each replica set is responsible for a subset of the database, known as a **shard** (this is a **horizontal partitioning** scheme)

- Query is issued to a router (mongos) instance
- With config server, mongos determines which shards to query
- Query sent to relevant shards (**partition pruning**)
- Shards run query on their data, send results back to mongos
- mongos merges the query results and returns the merged results to the application

Partition Pruning:

- Read/Write: for a query based on a
 - shard key**, the config server can determine that query only needs to go to one shard (contains that value of the shard key).
 - key other than the shard key, query will go to all shards
- Insert Query: Only relevant to 1 shard and only sent to that shard
 - The document to be inserted must have a value of the partition key (if not, it is an error)

Replication: Standard configuration $\Rightarrow$ 1 primary, 2 secondaries

Writes:

- The “Primary” receives all write operations

- Records writes onto its “operation log”
 - Secondaries will then replicate this operation log, apply it to their local copies of the data (thus ensuring data is synchronized), then acknowledge the operation
- Reads:
- The user can configure “read preference” to decide whether can read from secondaries (default), or the primary
 - Allowing reading from secondaries can **decrease latency and distribute load (improving throughput)**, but allows for reading stale data (it has only “eventual consistency”)

Wide Column Stores

- Example: BigTable, Cassandra, HBase
- Suitable for **massive-scale data** requiring **fast writes**: especially **time-ordered** data like logs, metrics, sensor data, etc.
- Column Families** are groups of related columns
- Sparsity**: if a column is not used for a row, it doesn't use space
- Efficiently operate on columns / subsets of columns (vs. doc. stores)

Graph Database

- Example: HyperGraphDB
- Useful if certain use-cases crucially involve **relationships**
 - recommending courses/study groups to students
 - managing complex prerequisites / time conflicts between courses to recommend a timetable

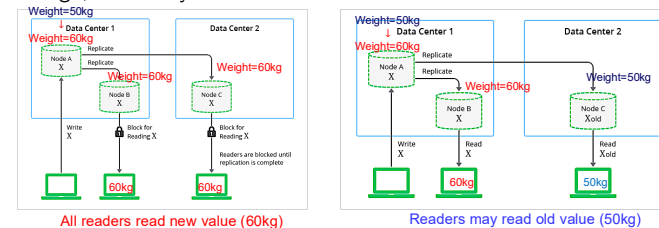
Vector Database

- Store vectors, each row represents a point in d dimensions (usually dense, numerical, and high-dimensional)
- DB features: scalability, real-time updates, replication
- Allow fast similarity search, i.e., given a query, retrieve similar neighbors from the database.
- similarity search algorithms used in vector databases is locality-sensitive hashing (LSH) (closely related to min-hashing)
- Examples:** Milvus, Redis, Weaviate, MongoDB Atlas

Consistency

Strong consistency: any reads immediately after an update must give the same result on all observers

Eventual consistency: if the system is functioning and we wait long enough, eventually all reads will return the last written value



ACID and BASE: R-DBMS provide **stronger** (ACID) guarantees, but many NoSQL system relax this to **weaker** BASE approach:

- Basically Available:** basic reading and writing operations are available most of the time (highly desirable)
- Soft State:**
 - Achieve **low latency**
 - Change with inputs until reach **eventual consistency**

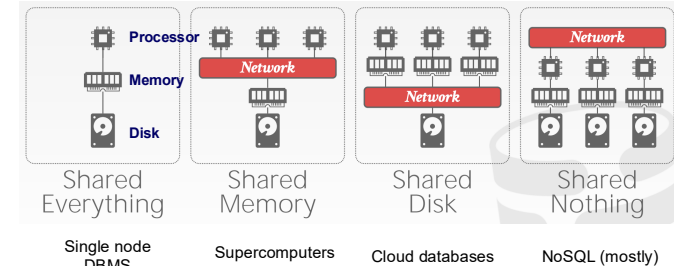
- Without guarantees, only have some prob. of knowing the state at any time
- Eventually Consistent:** Eventually become consistent
 - E.g. multiple reads eventually return the same value
 - Contrast to “strong consistency”
 - Acceptable for some applications (e.g. statistical queries, social network feed) but not for others (e.g. financial transactions)

Duplication

NoSQL usually uses Duplication (denormalization) rather than joining.

- Storage is cheap, duplicate data to improve efficiency.
- Tables are designed around the queries we expect to receive. (all necessary info. available in a single table / document)
- Beneficial esp. when we need to process a fixed type of queries.
- Leads to new problem, what if user changes their name $\rightarrow$ needs to be propagated to multiple tables

Distributed Databases



- Scalability:** allow database sizes to scale simply by adding nodes
- Availability/Fault Tolerance:** One fails, others can serve requests
- Latency:** Each request is served by the closest replica, **reducing latency**, particularly when globally distributed

Assumptions

- Nodes can go down $\rightarrow$ require sufficient redundancy
- Network connections between some nodes may fail
 - In some cases, connection failures cause network to split into 2 parts $\rightarrow$ cut off from each other. (**partition**)
- All nodes in a distributed database are **well-behaved** (follow some protocol, not ‘adversarial’ or trying to corrupt the database)

CAP Theorem

A distributed system **cannot** have all 3 of

- Consistency:** Every read receives the most recent write or an error. $\approx$ Strong Consistency. ($\neq$ ‘C’ in ACID)
- Availability:** Every request to a node must lead to a response.
- Partition Tolerance:** The system continues to operate even when network failures cause a partition into sub-networks.

Conclusion

- When **partitions**, systems have to sacrifice either Consistency (NoSQL often) or Availability (RDBMS often)

Data Partitioning

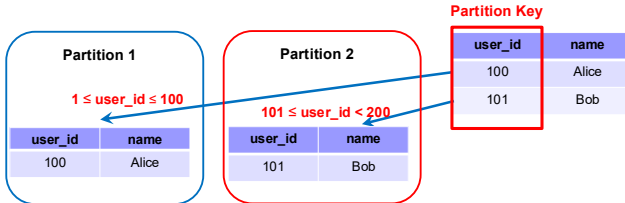
- Table Partitioning:** Put different tables on different machines
 - scalability:** each table **cannot** be split across multiple machines
- Horizontal Partitioning:** Different tuples stored in different nodes
 - Also called “sharding”
 - Partition Key/Shard Key:** Variable used to decide which node each tuple will be stored on.

- If we often need to filter tuples based on a column, or “group by” a column, then that column is a **suitable** partition key

★ **Summary** Key Partition, when is this good / bad:

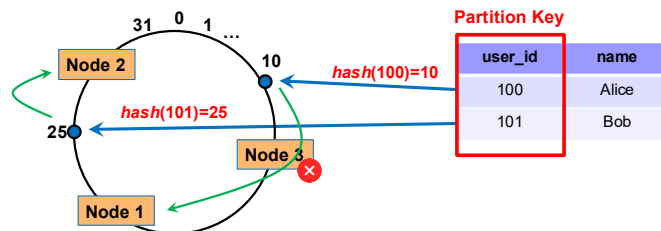
- Allow more efficient query / analysis on the data with same key.
- Data may be **skewed**. If certain key generates too many data, one partition will contain too many records, leading to poor parallelization and lack of scalability.
- Similar problems would occur if some key have too **high frequencies**. These can be mitigated by using **composite keys**.

- Range Partition:** split partition key based on range of values



- Allow efficient query and analysis within a **specific range**
 - Can lead to **imbalanced** shards $\Rightarrow$ Splitting the range is auto. handled by a balancer (tries to keep shards balanced) $\Rightarrow$ may become overloaded, causing inefficiency as the entire system is bottlenecked by this machine
 - Within a specific range, all data written to a specific partition, leading to the bottleneck issue at this specific partition / server.
- Hash Partition:** Hash partition key $\Rightarrow$ divide based on ranges
 - Spread out the data evenly among different partitions / servers
 - A query and analysis on data within a specific range may need to be sent to all partitions / servers
 - Potential Cons: Uneven distribution / some data correspond to a disproportionately large fraction of some keys

Consistent Hashing:



Partition: Each node has a ‘marker’ (rectangles) $\rightarrow$ Each tuple is placed on circle $\rightarrow$ assigned to node that comes clockwise-after it

Delete: Reassign all its tuples to node clock-wise after this one

Simple Replication: replicate a tuple in the next few additional nodes clockwise after the primary node used to store it

Multiple Markers: For each tuple, still assign it to the marker nearest to it in the clockwise direction.

- When remove a node, its tuples will not all be reassigned to the same node $\Rightarrow$ better balance load

Indexing

- Efficient way to access data in a database.
- Speed up the query, with the help of a query planner
- Local Indexing:** build a separate index in each shard.

- In MongoDB, the data is always indexed by the `_id` column, and optionally by other user-specified columns

Spark

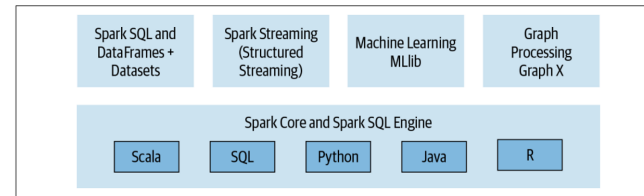
Issues with Hadoop Mapreduce:

- Network and disk I/O costs:** intermediate data has to be written to local disks and shuffled across machines, which is slow
- Not suitable for iterative (i.e. modifying small amounts of data repeatedly) processing: each individual step has to be modelled as a MapReduce job.

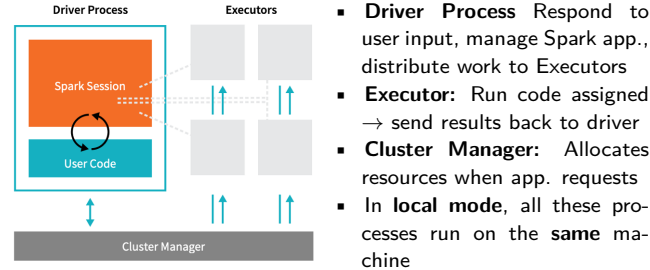
Spark Advantage

- Stores most of intermediate results in memory $\rightarrow$ much faster (esp. for iterative processing), when insufficient $\rightarrow$ spills to disk.
- Easier to program compared to Hadoop

Components and API Stack



Architecture

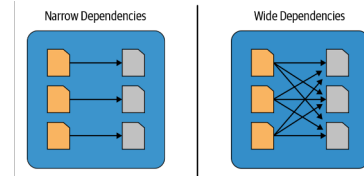


- Driver Process** Respond to user input, manage Spark app., distribute work to Executors
- Executor:** Run code assigned $\rightarrow$ send results back to driver
- Cluster Manager:** Allocates resources when app. requests
- In **local mode**, all these processes run on the **same machine**

Resilient Distributed Datasets (RDDs)

```
1 # Create an RDD, distributed over 3 partitions
2 dataRDD = sc.parallelize(["A", "B", "C", "D"], 3)
3 # Create an RDD: length of str
4 # Each str -> length
5 listLen = dataRDD.map(lambda s: len(s))
6 # Action retrieve all elem. of RDD to driver node.
7 listLen.collect()
```

- Immutable:** cannot be changed once created
- Transformations:** transform RDDs to RDDs
 - Lazy $\rightarrow$ not executed until being called $\Rightarrow$ optimize query plan to $\uparrow$ speed (e.g. removing unneeded operations)
 - Narrow Dependencies:** Each partition of the parent RDD is used by at most **one** partition of the child RDD
 - * map, flatMap, filter, contains
 - Wide Dependencies:** Each partition of parent RDD is used by **multiple** partitions of the child RDD
 - * reduceByKey, groupBy, orderBy, repartition



- Actions** trigger Spark to compute a result from a series of transformations. (show, count, save, collect)
- RDDs are actually **distributed** across machines.
- Transformations and Actions are executed in **parallel**.
- The results are only sent to the driver in the final step.

```
1 # Reads file on each worker node in parallel
2 # Not driver node
3 textFile = sc.textFile("File.txt")
4 linesWithSpark = textFile.filter(
5     lambda line: "Spark" in line
6 )
7 linesWithSpark.count() # send result back to driver
8 linesWithSpark.first() # send result back to driver
```

Caching

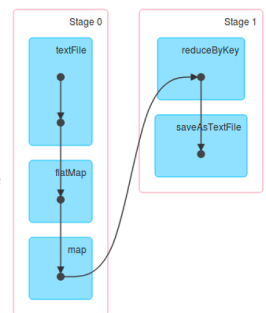
```
1 lines = sc.textFile("hdfs://...")
2 errors = lines.filter(lambda s: s.startswith("ERR"))
3 messages = errors.map(lambda s: s.split("\t")[2])
4 messages.cache() # caching operation
5 messages.filter(lambda s: "mysql" in s).count()
6 messages.filter(lambda s: "php" in s).count()
```

- cache(): saves an RDD to memory of each worker node
- persist(options): save RDD to memory/disk/off-heap memory
- Cache when expensive to compute, or needed multiple times.
- If worker nodes have insufficient memory, they evict LRU RDDs.
- Tips:** aware of memory limitations when caching.

Directed Acyclic Graph (DAG)

```
1 val file = sc.textFile("hdfs://...")
2 val counts = file.flatMap(line => line.split(" "))
3     .map(word => (word, 1))
4     .reduceByKey(_ + _)
5 counts.save("...")
```

- Internally, Spark creates a DAG which represents all the RDD objects and how they will be transformed
- Transformations construct this graph
- Actions trigger computations on it
- Consecutive narrow dependencies are grouped together as **stages**.
- Within** stages, Spark performs consecutive transformations on **same** machines.



- Across stages**, data needs to be shuffled, i.e. exchanged across partitions, in a process very similar to map-reduce, which involves writing intermediate results to disk
- Minimizing shuffling is good practice for improving performance.

Lineage and Fault Tolerance

- Spark does not use replication to allow fault tolerance.

Reason: Spark tries to store all the data in memory → capacity much more limited than disk → duplicating all data is expensive.

- Lineage approach:** worker node goes down → replace it by a new worker node → use DAG to **recompute** data in lost partition only.

DataFrames and Datasets

- A higher level interface than RDD. Recommend to use than RDD.
- Has transformations that resemble SQL operations
- All DataFrame operations are still ultimately compiled down to RDD operations by Spark.
- DataFrame is just an alias for Dataset[Row]

DataFrames and RDD

RDD	DataFrame
Instruct Spark how to compute the query	Tell Spark what to do, instead of How to do
The intention is opaque to Spark	Far more expressive & simpler - Using a Domain Specific Language - Use high-level DSL operators to compose the query
Spark also does not understand the structure of the data in RDDs or the semantics of user functions	Spark can inspect/parse this query and understand intention → optimize/arrange operations for efficient execution

DataFrames: expression based compared to functional based in RDDs (our code affects the efficiency of the program), we just need to tell Spark what we want it to do:

```
1 # Reads in a DataFrame from a CSV file.
2 flightData2015 = spark.read\
3 .option("inferSchema", "true")\
4 .option("header", "true")\
5 .csv("/mnt/deqg/flight-data/csv/2015-summary.csv")
6 # Action: sorts by count and output the first 3 rows
7 flightData2015.sort("count").take(3)
8 # Output: Array([United States,Romania,15], [United States,Croatia...
```

Use SQL queries to transform DataFrames. This takes in a DataFrame and returns a DataFrame (the output of the query).

```
1 flightData2015.createOrReplaceTempView("
  flight_data_2015")
2 maxSql = spark.sql("""
3   SELECT DEST_COUNTRY_NAME, sum(count) as
  destination_total
4   FROM flight_data_2015
5   GROUP BY DEST_COUNTRY_NAME
6   ORDER BY sum(count) DESC
7   LIMIT 5
8   """)
9 maxSql.collect()
```

Exactly same query

```
1 from pyspark.sql.functions import desc
2 flightData2015.groupBy("DEST_COUNTRY_NAME")\
3 .sum("count")\
4 .withColumnRenamed("sum(count)",
5   "destination_total")\
6 .sort(desc("destination_total"))\
7 .limit(5)\
8 .collect()
```

DataFrame API

- groupBy()

```
1 df.groupBy("id")
2 .agg(sum("amount").alias("total")).show()
```

is equivalent to

```
1 SELECT id, SUM(amount) AS total FROM df
2 GROUP BY customerId;
```

- join(): Consider

```
1 tableA.join(tableB, on="key", how="inner")
```

There will be data shuffle during the execution, rows with same key will be sent to same partition. Therefore the final table will show in "cluster" of same key

- Window local aggregation ⇒ Rank by key Example

```
1 window = Window.partitionBy("key").orderBy(col("amt
  ").desc())
2 ranked_tb = tb.withColumn("key", row_number().over(
  window))
```

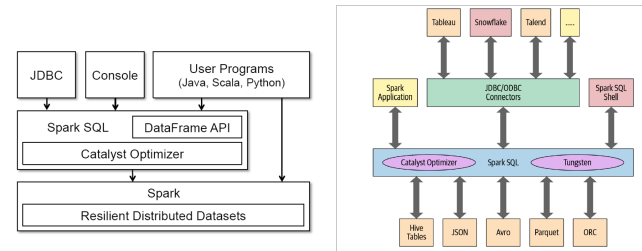
Datasets

```
1 case class Flight(DEST_COUNTRY_NAME: String,
  ORIGIN_COUNTRY_NAME: String, count: BigInt)
2 val flightsDF = spark.read.parquet("/mnt/deqg/flight-
  data/parquet/2010-summary.parquet/")
3 val flights = flightsDF.as[Flight]
4 flights.collect()
```

- Dataset flights is **type safe**: its type is the Flight class.
- Now when calling collect(), it will also return objects of the Flight class, instead of Row objects.

Spark SQL

- Unifies Spark components and permits abstraction to DataFrames/Datasets in Java, Scala, Python, and R
- Keep track of schema and support optimized relational operations



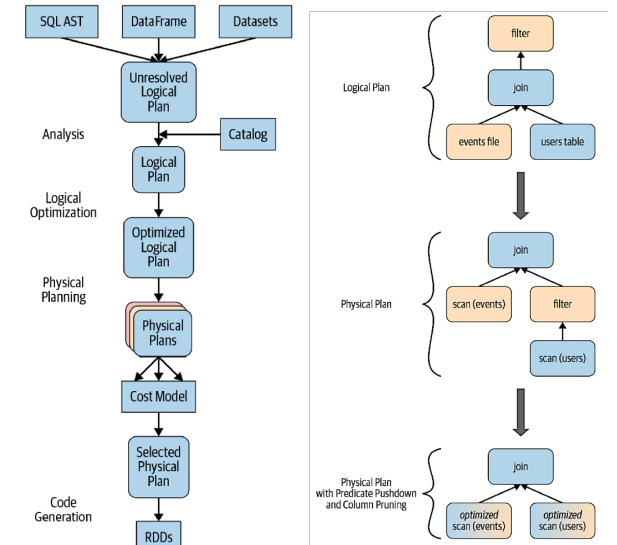
Catalyst Optimizer: Query optimization **framework** takes computational query and converts it into an execution plan through four transformational phases

- Analysis
- Logical optimization
- Physical planning
- Code generation (Project Tungsten)

- Primary purpose:** Optimize logical and physical query plans generated by Spark's DataFrame and SQL APIs

```
1 // In Scala
2 // Users DataFrame read from a Parquet table
3 // Events DataFrame read from a Parquet table
4 val eventsDF = ...
5 // Join two DataFrames
6 val joinedDF = users
```

```
7 .join(events, users("id") === events("uid"))
8 .filter(events("date") > "2015-01-01")
```



Project Tungsten:

- Substantially improve the memory and CPU efficiency of Spark applications
- Push performance closer to the limits of modern hardware

Bottleneck Analysis Example

The traceA dataframe keeps logs, and each log entry represents the log info. of one web page access, including various fields: ip, IP address of log entry and **time** (the amount of time of that access)

```
1 maxsql = spark.sql("""
2   SELECT ip, sum(time) as access_total
3   FROM traceA
4   WHERE time > 0.1
5   GROUP BY ip
6   ORDER BY sum(time) DESC
7   """)
8 maxsql.collect()
```

- If the dataframe is not entirely in memory, reading traceA (L2-4) may incur potential I/O cost.
- Size of data generated (L4-6): Both operations are wide transformations ⇒ require data shuffling through network.
- If grouped data after L5 is highly skewed, L6 will have task straggler issues (some tasks requiring ≫ time than others).
- L7 may return too many results ⇒ can add LIMIT to SQL

Typical Machine Learning Pipeline

Preprocessing → Training → Testing

→ Evaluation

Data Quality: Missing Values

- Missing at random: missing values are randomly distributed.

If data is instead missing **not at random** then the **missingness** itself may be important information.

- Information was not collected
- How to handle

- Eliminate objects (rows) with missing values
- Fill in the missing values ("imputation") by mean/median
- Fitting a regression model to predict given other attributes
- Dummy variables: optionally insert a column which is 1 if the variable was missing, and 0 otherwise
- Use PySpark to address missing value (default strategy: mean)

```
1 from pyspark.ml.feature import Imputer
2 imputer = Imputer(
3     inputCols=["a", "b"],
4     outputCols=["out_a", "out_b"]
5 )
6 model = imputer.fit(df)
7 model.transform(df).show()
```

Categorical Encoding & One Hot Encoding: Numerical values are often assigned in a way that represents the ordinal relationship or inherent order among the categories.

Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}} : \mathbb{R} \rightarrow (0, 1)$.

- $\sigma(-z) = 1 - \sigma(z)$
- $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \sigma(z)\sigma(-z)$
- $\sigma''(z) = \sigma(z)(1 - \sigma(z))(1 - 2\sigma(z))$
- $g(x) = -\log \sigma(x) \Rightarrow g''(x) = \sigma(x)(1 - \sigma(x)) \geq 0 \Rightarrow \text{convex}$

Parallel Gradient Descent

Evaluate

1. Classification:

- **Accuracy:** $\frac{TP+TN}{TP+FN+FP+TN}$
- **Precision:** $P = \frac{TP}{TP+FP}$
- **Recall:** $R = \frac{TP}{TP+FN}$
- **F1 Score:** $F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$
(Hamonic mean of P and R)
- **Maximize Precision/Recall**
 $\Leftrightarrow$ **Minimize FP/FN**

	Actual Label	
	Positive	Negative
Predicted Label		
Positive	TP	FP
Negative	FN	TN

2. Regression:

- $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- $RMSE = \sqrt{\sum_{i=1}^n (y_i - \hat{y}_i)^2}$
- $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **R Squared Value:** The percentage of variance from the **samples** can be explained through the model (closer to 1, better)

Pipelines in Spark ML

Spark ML pipelines chains multiple stages of data processing and modeling:

- **Why:** Pipelines allow better code reuse, and easier to perform cross validation, hyperparameter tuning (easier to treat as encapsulated).
- **Transformers:** Map DataFrames to DataFrames $\Rightarrow$ output a new DataFrame which append their result to the original DataFrame
 - e.g. one-hot encoding, normalization, model predictions
 - `transform()` performs its transformation
 - **Fitted model is a Transformer** that transforms a DataFrame into one with the predictions appended.
- **Estimators:** Algo. takes in data, produces a fitted model
 - `fit()` method $\Rightarrow$ returns a Transformer

```
1 from pyspark.ml.classification import
   LogisticRegression
2 training = spark.read.format("libsvm").load("data/
   mllib/sample_libsvm_data.txt")
```

```
3 lr = LogisticRegression(maxIter=10)
4 lrModel = lr.fit(training)
5 print("Coefficients: " + str(lrModel.coeficients))
6 print("Intercept: " + str(lrModel.intercept))
```

- **Pipeline:** Combines transformers and estimators into a single workflow.
- Pipeline chains multiple Transformers and Estimator to form ML workflow.

Pipeline Execution

```
1 pipeline = Pipeline(stages=[...])
2 pipelineModel = pipeline.fit(train_df)
```

- **Training Time:** `Pipeline.fit()` called:
 - Transformers execute their `transform()` methods.
 - Estimators execute their `fit()` methods to produce fitted models.
- **Test Time:** Output of `Pipeline.fit()` is a `PipelineModel`, which is a Transformer. Calling `PipelineModel.transform()` applies the transformations and predictions to new data.

Stream

Batch/Offline: Operate on the full dataset

Stream/Online: Process their input as it is received

- Potentially infinite
- System cannot store entire stream accessibly (limited memory)
- Input elements enter at a rapid rate from input ports
- Elements of the stream are sometimes referred to as tuples
- In many real settings, data is high volume and constantly arriving over time

Stateful Stream Processing

- not just perform trivial record-at-a-time transformations
- ability to store and access intermediate data
- state can be stored and accessed in many different places (program variables/local files/embedded/external DB)

Micro-Batch Stream Processing (e.g. Spark Structured Streaming)

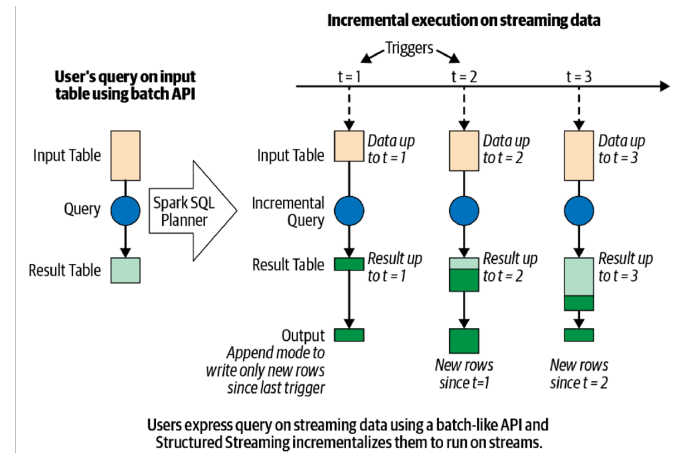
- divides the data from the input stream into micro batches
- each batch processed in the Spark cluster in a distributed manner
- small deterministic tasks generate output in micro-batches
- **Advantages**
 - quickly and efficiently recover from failures
 - **deterministic** nature ensures **end-to-end exactly-once** processing guarantees
- **Disadvantages:**
 - latencies of a few seconds (usually OK for many applications)
 - Application may incur more than a few seconds delay in other parts of pipeline

General Idea:

- A single, unified programming model and interface for batch and stream processing
- A broader definition of stream processing

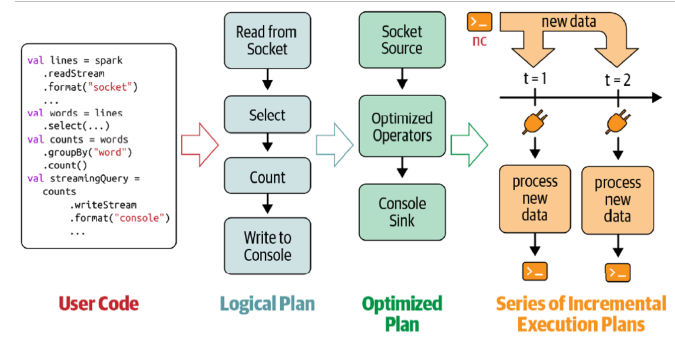
Structured Stream Model:

- Data stream as an unbounded table
- New records in data stream $\Leftrightarrow$ New rows append to unbound table. (**exactly once**, even when system failures or errors)



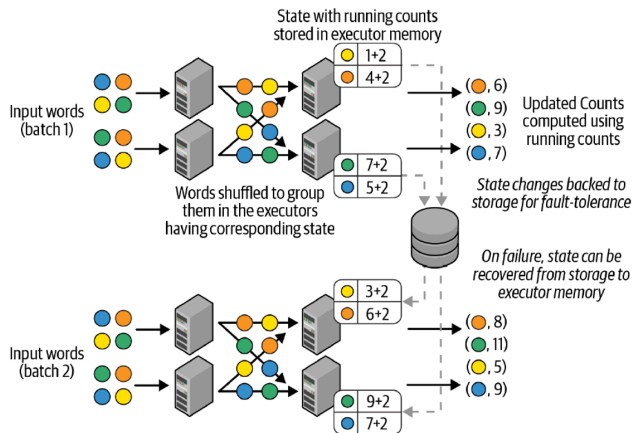
Define a Streaming Query

1. Define input sources
2. Transform data
3. Define output sink and output mode
 - Output writing details
 - where and how to write the output
4. Specify processing details
 - Triggering details: when to trigger the results and processing of newly available streaming data.
 - Checkpoint Location: store the streaming query process info for failure recovery
5. Start the query



Data Transformation

- **Stateless Transformation**
 - Process each row individually without needing any info. from previous rows
 - Projection: `select()`, `explode()`, `map()`, `flatMap()`
 - Selection: `filter()`, `where()`
- **Stateful Transformation**
 - A simple example: `DataFrame.groupBy().count()`
 - In every micro-batch, incremental plan adds count of new records to previous count generated by previous micro-batch
 - The partial count communicated between plans is the state
 - The state is maintained in the memory of Spark executors and is checkpointed to the configured location to tolerate failures.



Stateful Stream Aggregation: Aggregations Not Based on Time

- Global aggregations

```
1 runningCount = sensorReadings.groupBy().count()
```

- Grouped aggregations

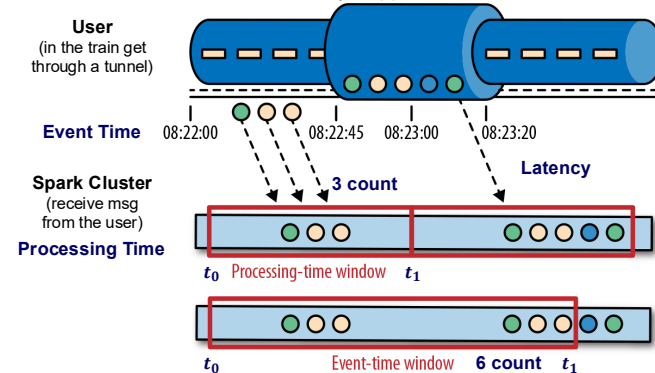
```
1 mean = sensorReadings.groupBy("Id").mean("val")
```

- All built-in aggregation functions in DataFrames are supported

Time Semantics

Processing Time: time of stream processing machine

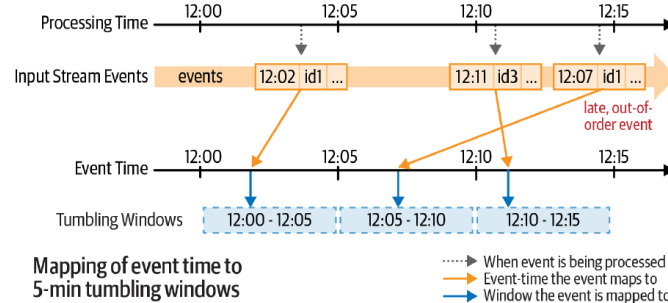
Event Time: time an event actually happened



- Event time completely decouples the processing speed from the results.
- Operations based on **event time** are predictable and results are **deterministic**.
- An event time window computation will yield the same result no matter how fast the stream is processed or when the events arrive at the operator.

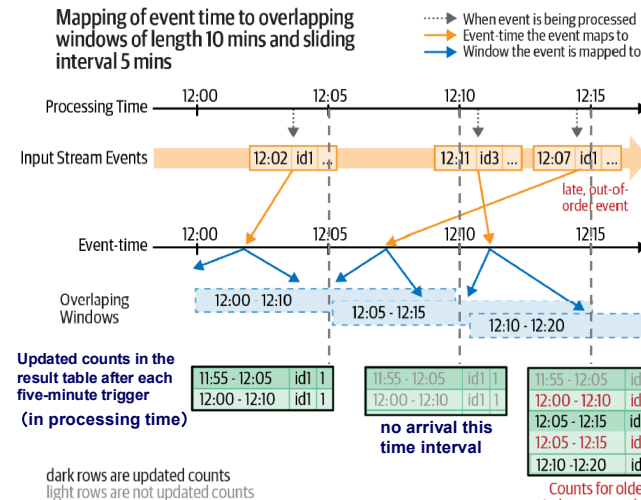
Tumbling Windows: No overlap windows

```
1 (sensorReadings.groupBy(
2   "sensorId",
3   window("eventTime", "5 minute")
4 ).count())
```



Sliding Window: Overlapping window length

```
1 (sensorReadings.groupBy(
2   "sensorId",
3   window("eventTime", "10 minute", "5 minute")
4 ).count())
```

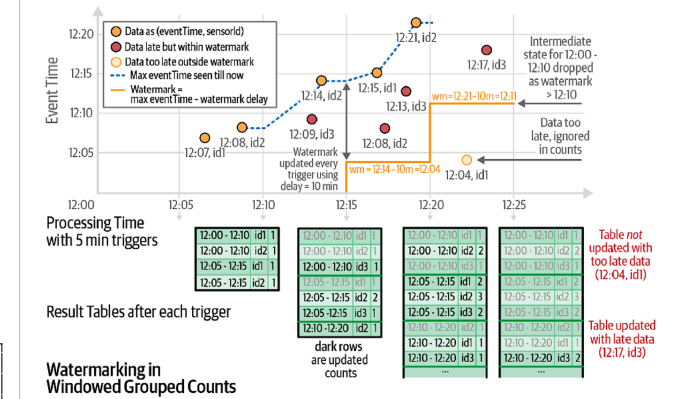


Watermarks: watermark = max event time seen - delay (10min here)

```
1 (sensorReadings
2   .withWatermark("eventTime", "10 minute")
3   .groupBy(
4     "sensorId",
5     window("eventTime", "10 minute", "5 minute")
6   ).count())
```

- Consider time window $[t_{start}, t_{end}] \Rightarrow$ **drop** window with $t_{end} <$ watermark and only **update** window with $t_{end} >$ watermark

Summary Justify Your Choice. From 8:15 to 8:30, watermark = 8:14 - 10 min = 8:04 > 8:00, thus TransactionTime window 7:30-8:00 can be dropped from memory and will not be updated anymore. The rest TransactionTime windows are updated per normal



Performance Tuning: Besides tuning Spark SQL engine, consider

- Cluster resource provisioning appropriately to run 24/7
- # of partitions for shuffles to be set much **lower** than batch queries
- Setting source rate limits for stability
- Multiple streaming queries in the same Spark application

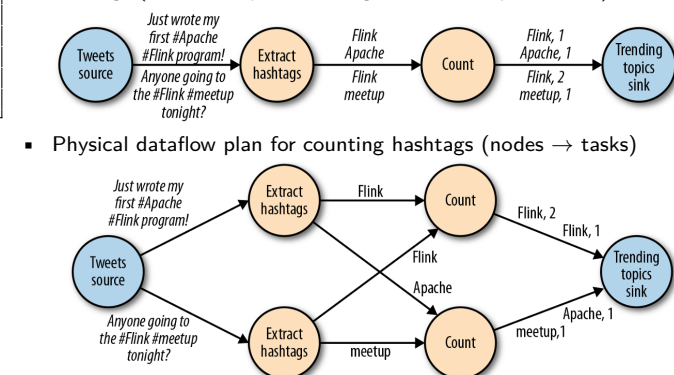
Flink

Spark	Flink
Microbatch streaming processing	Real-time streaming processing
Seconds latency	Milliseconds latency
Checkpoints each microbatch (synchronous)	Checkpoints distributedly (asynchronous)
Watermark: a configuration to let user determine how to handle the late event	Watermark: a special record to let user estimate the current event time (Late handling functions + watermarks to handle the late events)

- continuously track and analyze patterns of data over time
- detect anomalies and fraud patterns as they occur
- powerful tool for applications demand both speed and real-time stateful analysis

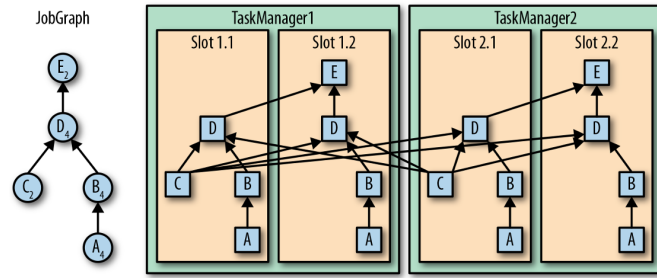
Dataflow Model

- Dataflow Graph: A logical dataflow graph to continuously count hashtags (nodes $\rightarrow$ operators; edges $\rightarrow$ data dependencies)



Flink: Distributed system for stateful parallel data stream processing.

Task Execution

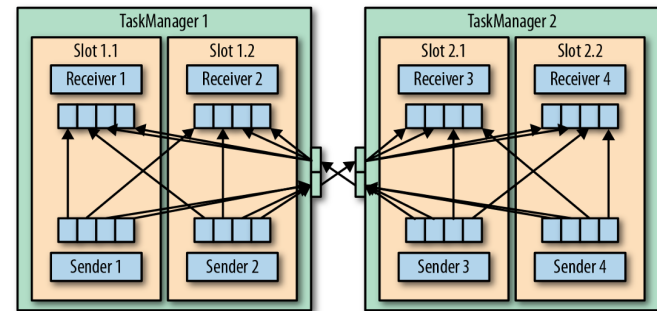


Task Manager: Offers a certain # of **processing slots** to control the # of tasks it is able to concurrently execute.

- Can execute several tasks at the same time
 - tasks of the same operator (data parallelism)
 - tasks of different operators (task parallelism)
 - tasks from a different application (job parallelism)

Processing Slot: Execute one slice of an application (one parallel task of each operator of the application)

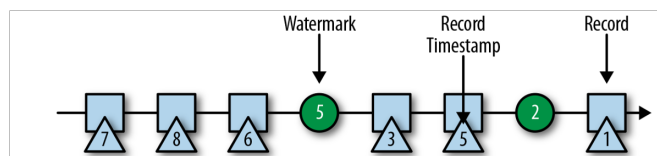
Data Transfer



- The tasks of a running app. are **continuously exchanging data**.
- The TaskManagers take care of shipping data from sending tasks to receiving tasks asynchronously. (ideally in the same slot A→B))
- The network component of a TaskManager collects records in buffers before they are shipped.

Event Time Processing

- Every record must be accompanied by an **event timestamp**
- Flink event-time application must also provide **watermarks**
 - indicate the progress of event time
 - used to derive the current event time at each task
 - implemented as special records holding a timestamp (Long)
 - flow in a stream of regular records with annotated timestamps
 - watermark t_n means record before event time t_n should be processed alr.
 - help system understand when it can trigger event time-based operations, such as window functions



State Management

State Backend

- Local State Management**
 - a task of a stateful operator typically reads and updates its state for each incoming record.
 - each parallel task locally maintains its state in memory (faster state accesses)
- Checkpointing**
 - A TaskManager process may fail at any point, hence its storage must be considered **volatile**
 - Checkpoints the state of a task to a remote/persistent storage
 - The remote storage for checkpointing could be a distributed filesystem or a database system (e.g., HDFS, S3)

Flink's Checkpointing Algorithm

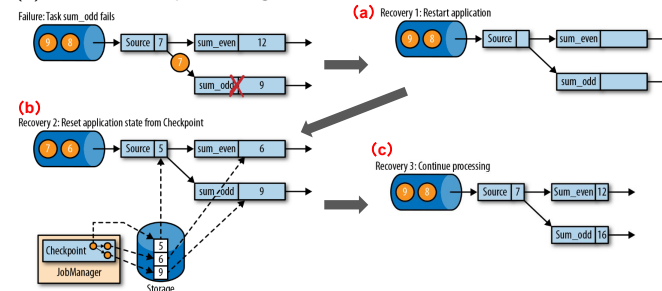
- based on the Chandy-Lamport algorithm for distributed snapshots
- not pause app. but decouples checkpointing from processing
- some tasks continue processing while others persist their state
- uses a special type of record called a **checkpoint barrier**
- periodically captures **only** the state of the computation
- ensuring exactly-once semantics through careful mgmt. of state

Checkpoint Barrier:

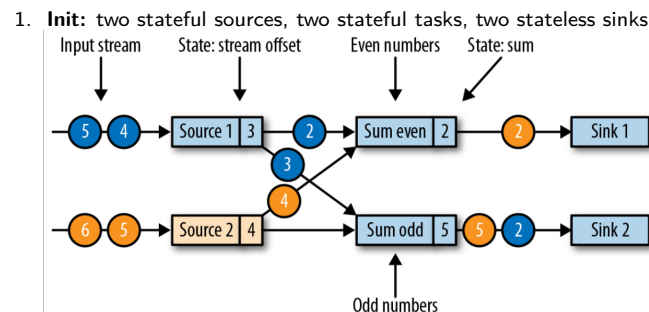
- Injected by source operators into **regular** stream of records
- cannot overtake or be passed by other records
- carries a checkpoint ID to identify ckpt it belongs to
- logically splits a stream into two parts
- All state modifications due to records that precede a barrier are included in the barrier's checkpoint and all modifications due to records that follow the barrier are included in a later checkpoint.

Failure Recovery

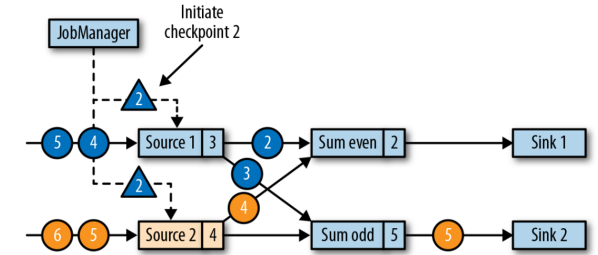
- Restart the whole application.
- Reset the states of all stateful tasks to the latest checkpoint.
- Resume the processing of all tasks.



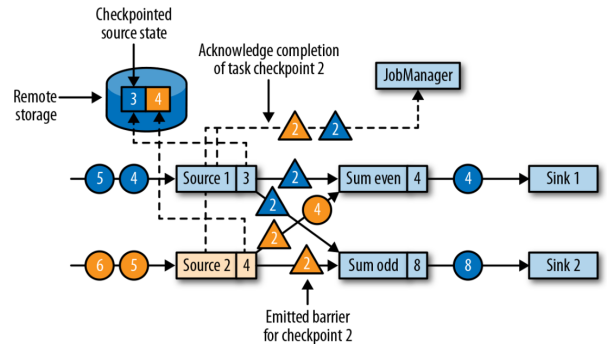
Checkpointing Process



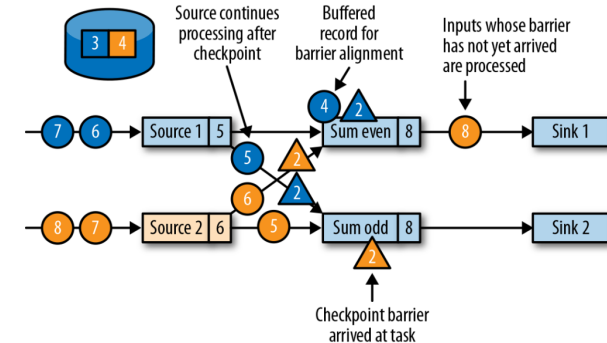
- JobManager initiates a checkpoint by sending a message to all sources



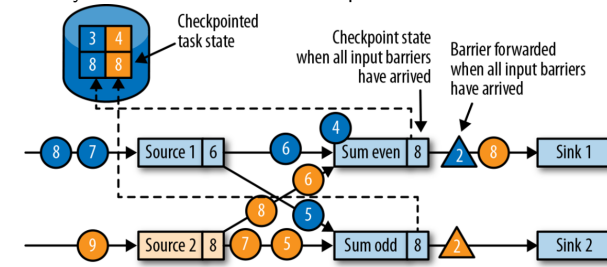
- Sources checkpoint their state and emit a checkpoint barrier



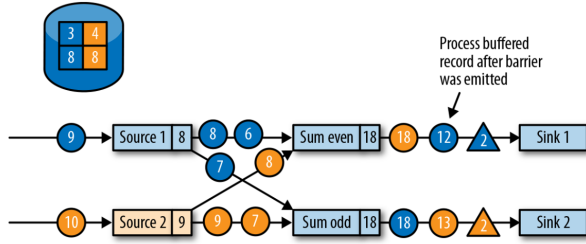
- Tasks wait to receive a barrier on each input partition
 - Task received barrier will buffer the following income stream for barrier alignment.
 - All other records are regularly processed



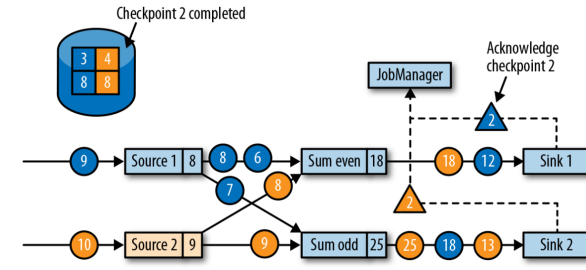
- Tasks checkpoint their state once all barriers have been received, then they ack. and forward the checkpoint barrier



- Tasks continue regular processing after the checkpoint barrier is forwarded



- Sinks acknowledge the reception of a checkpoint barrier to the JobManager
 - a checkpoint is complete when all tasks have acknowledged the successful checkpointing of their state



Graph

Page Rank

- Measuring the importance of pages is necessary for many web-related tasks.
- PageRank-like methods are also used in many other applications (bioinformatics)

Modeling: Node: Webpage; **Edge:** Hyperlink

Version 1: Links as Votes

Idea: Page is more important if it has more in-links

Problem: Malicious user can create a huge number of 'dummy' web pages, to link to their one page, to drive up its rank!

Solution: Let # of votes a page has **proportional** to its own importance.

- Low importance contribute little votes
- Links from important pages count more → recursive
- Main idea of PageRank** ⇒ recursively defines the importance of a page based on the importance of the pages linking to it

Formulation:

- For each page j , define its "importance" (or rank) as r_j
- If page j with importance r_j has n out-links, each link gets r_j/n votes (Here we use d_i denote n)
- Page j 's own importance is the sum of the votes on its in-links

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

- Additional constraint forces uniqueness

$$\sum_{i \in S} r_i = 1$$

Transition Prob. Matrix: Attribute $M_{ji} = P(i \rightarrow j)$ **Different Here!**
Flow Equations:

$$\mathbf{r} = \mathbf{M} \cdot \mathbf{r}$$

Power Iteration Method: Suppose there are N web pages

- Initialize: $\mathbf{r}^{(0)} = [1/N, \dots, 1/N]^T$
- Iterate: $\mathbf{r}^{(t+1)} = \mathbf{M} \cdot \mathbf{r}^{(t)}$
- Stop when $|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}| < \epsilon$

Dead End: node has no out-links

- Random walk has nowhere to go to
- Such pages cause importance to leak out

Spider traps: All out-links are within the group

- Random walk gets stuck in a trap
- eventually absorb all importance
- One node:** the node has one out-link pointing to itself
- Multiple nodes:** needs to be a proper subset of the graph

Teleports

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + \underbrace{(1 - \beta) \frac{1}{N}}_{\text{Teleport term}}$$

Solution(Spider Traps): Each step, random surfer has two opt.

- With prob. β , follow a link at random
- With prob. $1 - \beta$, jump to some random page
- Never get stuck in a spider trap by teleporting out of it in a finite number of steps

Solution(Dead End)

- If at a Dead End, Always Teleport (prob. 1) ⇒ change $\mathbf{M}$ by making m connected to every node (**including itself**).
- Make matrix column stochastic

Advantage:

- Helps random walker to occasionally jump to less popular/impt nodes ⇒ spread out importance factors among all nodes.
- Helps even if **no** dead end/spider trap
- Helps the algorithm to converge **faster**

Convergence: Evaluate by MAE, lower MAE ⇔ Faster Converge

Google Matrix:

$$\mathbf{A} = \beta \mathbf{M} + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$$

Topic-Specific PageRank

- Goal:** Evaluate Web pages not just according to their popularity, but by **how close** they are to a particular topic
- Allows search queries to be answered based on interests of the user

Teleport can go to:

- Standard:** Any page with equal probability
⇒ To avoid dead-end and spider-trap problems
- Topic Specific:** A topic-specific set of **relevant** pages

Idea: Bias the random walk

- When random walker teleports, it picks a page from a set S
- S contains only pages that are relevant to the topic
- For each teleport set S , we get a different vector $\mathbf{r}_S$

Matrix Formulation:

$$\mathbf{A}_{ij} = \begin{cases} \beta \mathbf{M}_{ij} + (1 - \beta)/|S| & \text{if } i \in S \\ \beta \mathbf{M}_{ij} + 0 & \text{otherwise} \end{cases}$$

- Maintains sparseness

- $\mathbf{A}$ is stochastic

Summary Connection: Flow Equations and Matrix Equations are equivalent to each other. Since $R_a + R_b + R_c = 1$, both equations / solutions will get the same values for R_a , R_b , R_c

Pregel

Characteristics of Graph Algorithms

- Local computations at each vertex
- Passing messages to other vertex
- Algo. implemented from view of a single vertex, performing one iteration based on messages from its **neighbor**
 - compute(): describe one vertex behavior in one step.
 - Framework abstracts away scheduling/implementation details.

Sync. Computational Model: Consists of a series of **supersteps**

- Each** superstep, framework invokes a user-defined compute() for each vertex (conceptually in parallel)
- compute() specifies a single vertex v behavior at superstep s
 - It can **read** messages sent to v in superstep $s-1$;
 - It can **send** messages to other vertices that will be read in $s+1$;
 - It can **read** or **write** the value of v and the value of its outgoing edges (or even add or remove edges)
- Synchronization occurs at the end of each superstep (Asynchronous not supported).
- Termination:**
 - A vertex can choose to deactivate itself
 - Is "woken up" if new messages received
 - Computation halts when all vertices are inactive

Implementation: Master and Workers architecture

- Vertices are hash partitioned and assigned to workers by edge cut
- Each worker maintains state of its portion of graph **in memory**
- Computations happen **in memory**
- In each superstep, each worker apply compute() to all its vertices
- Messages** from vertices are sent, either to vertices on the same worker, or to vertices on different workers (buffered locally and sent as a batch to *reduce network traffic*)
- reduces the need for global coordination

Fault Tolerance

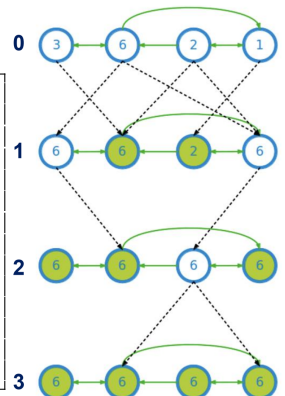
- Checkpointing to persistent storage
- Failure detected through heartbeats
- Corrupt workers are reassigned and reloaded from checkpoints

Computing Max Value

- Sends its current value to neighbors
- Each vertex updates its value (max)

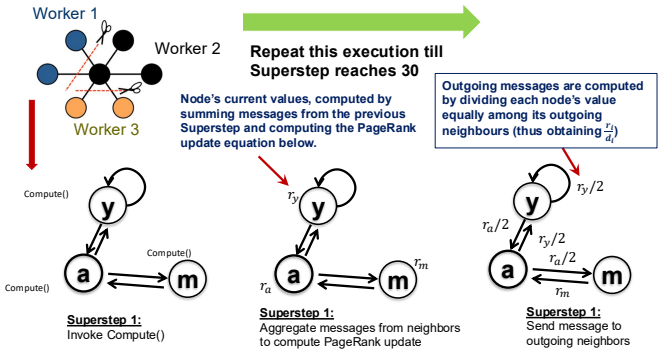
```

1 Compute(v, messages):
2   changed = False
3   for m in messages:
4     if v.getValue() < m:
5       v.setValue(m)
6       changed = True
7   if changed:
8     for each outneighbor w:
9       sendMessage(
10        w,
11        v.getValue()
12      )
13   else:
14     voteToHalt()
  
```



PageRank with Teleport

```
1 Compute(v, messages):
2   if getSuperStep() = 0:
3     v.setValue(1 / getNumVertices())
4   if getSuperStep() >= 1:
5     sum = 0
6     for m in messages:
7       sum += m
8     v.setValue(
9       0.15 / getNumVertices() + 0.85 * sum
10    )
11  if getSuperStep() < 30:
12    sendMsgToAllEdges (
13      v.getValue() / len(getOutEdgeIterator())
14    )
15  else:
16    voteToHalt()
```



Spark for Graphs

- Integration of record-oriented and graph-oriented processing
- GraphX**: Extends RDDs to Resilient Distributed Property Graphs
- Property Graphs**: Present different views of the graph (vertices, edges, triplets). Support distributed Pregel-like computation
- Graphframe**: Extends GraphX to provide a DataFrame API and support for Spark's different language bindings
- Spark uses **Vertex Cut** to partition the graph

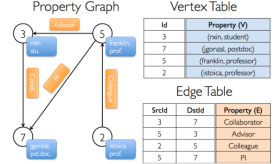
Stage 1: Join

```
CREATE VIEW triplets AS
SELECT s.Id, d.Id, s.P, e.P, d.P
FROM edges AS e
JOIN vertices AS s JOIN vertices AS d
ON e.srcId = s.Id AND e.dstId = d.Id
```

Vertices:

Edges:

Triplets:



Stage 2: Group-By

```
SELECT t.dstId, 0.15+0.85*sum(t.srcP*t.eP)
FROM triplets AS t GROUP BY t.dstId
```

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

MISC (Summary from PYP)

Platform Selection:

- Large batch-processing jobs: HDFS
- Real Time Random Data Access: NoSQL
- Real Time Stream Computing: Flink/Spark Stream